

Azure Storage & Databases — Hands-on Lab

Blob Storage (object storage) and Azure SQL Database

September 2026

Contents

Introduction & Setup	2
Hands-on Lab Exercises: Azure Storage and Databases	2
Part 1 — Lab Questions	6
Question 1 — Create a Storage Account and Container	6
Question 2 — Upload a File to Blob Storage	7
Question 3 — Lock Down the Storage Account	8
Question 4 — Optimize Cost with the Access Tier	9
Question 1 — Create a Logical Server and a Database	10
Question 2 — Configure the Server Firewall	11
Question 3 — Scale the Database	12
Question 4 — Serverless with Auto-Pause	13
Part 2 — Solutions	14
Question 1 — Create a Storage Account and Container	14
Solution — Create a Storage Account and Container	14
Question 2 — Upload a File to Blob Storage	16
Solution — Upload a File to Blob Storage	16
Question 3 — Lock Down the Storage Account	18
Solution — Lock Down the Storage Account	18
Question 4 — Optimize Cost with the Access Tier	20
Solution — Optimize Cost with the Access Tier	20
Question 1 — Create a Logical Server and a Database	22
Solution — Create a Logical Server and a Database	22
Question 2 — Configure the Server Firewall	24
Solution — Configure the Server Firewall	24
Question 3 — Scale the Database	26
Solution — Scale the Database	26
Question 4 — Serverless with Auto-Pause	28
Solution — Serverless with Auto-Pause	28
Part 3 — Practice Exercises	30
Practice Exercises — Storage and Databases	30
Part A — Blob Storage	31
Part B — Azure SQL Database	32

Introduction & Setup

Hands-on Lab Exercises: Azure Storage and Databases

Topic: Storing and serving data in Azure with **Blob Storage** (object storage) and **Azure SQL Database** (a managed relational database).

Welcome back. This is the second lab in the Cloud Administrator series. The first lab was about **compute** (Virtual Machines and App Service). This one is about **data**: where it lives, who can reach it, and what it costs to keep.

A real admin does not memorize every command. They know which breadcrumbs to follow. Each problem here is a short, real world scenario. Your job is to work out what it demands and solve it with the Azure CLI. You get a script to check your work and a full solution walkthrough, freely, because this is a low stakes place to build competency. Break things here so you do not have to break them in production.

What you will build

Eight graded questions across two categories, plus ten practice exercises:

- **Blob Storage:** create an account and container, upload a blob, lock down anonymous public access, and optimize cost with access tiers.
- **Azure SQL Database:** create a logical server and database, open the firewall so a client can connect, scale a database up, and move it to a serverless model that auto pauses when idle.

How this lab is different from the Compute lab

If you finished the Compute lab, the rhythm here is the same (one folder per question, `./setup.bash`, solve, `./validate.bash`, `./cleanup.bash`), but the subject matter brings several new ideas that compute did not:

- **Control plane versus data plane.** With a VM you mostly manage the resource itself. With storage and databases there are two layers: managing the resource (creating an account or a database) and working with the data inside it (uploading a blob, running a SQL query). They often use different tools and different permissions. You will feel this the first time an owner of the subscription still cannot read a blob without the right data plane access.
- **Globally unique names.** A VM name only has to be unique inside its resource group. A storage account name and a SQL server name become public DNS names, so they must be unique across all of Azure and all lowercase. Because of that, the validators discover the resource in your resource group rather than assuming a fixed name, and you are free to pick your own.
- **Resource providers.** Before a subscription can create a kind of resource, its provider must be registered. Storage is usually registered already; Azure SQL often is not, so the SQL setups register `Microsoft.Sql` for you the first time.
- **Connecting to your data.** For the SQL questions you actually connect to the database with `sqlcmd` and watch a query fail before a firewall rule and succeed after. That is a new muscle: the resource can be healthy and still unreachable until you open the door.

- **Security and cost as first class tasks.** You will make a public blob private and prove it with `curl`, and you will move data to cheaper tiers and a database to a serverless model. Securing and right sizing data is a large part of a real admin's week.
- **Observe, do not assume.** Every question asks you to see the effect with your own eyes: a `curl` that goes from readable to refused, a tier that flips from Hot to Cool, a connection that starts working. The validator confirms it, but you should see it first.

The three skills we are building

1. **Reading documentation.** The first question an admin asks is “where are the provider's docs for X?” Learn to navigate them and you can learn any service.
2. **Tooling.** We use the **Azure CLI** (`az`) exclusively. No portal for the exercises. We like the hard way, because the hard way is the way that sticks.
3. **Troubleshooting.** When things break, composure first. Understand the cause, fix it, and write down what happened.

Prerequisites

- **An Azure subscription**, paid (pay as you go). The labs use inexpensive resources (Standard_LRS storage and Basic or small SQL tiers).
- **A lab VM (jumpbox) to work from.** You created this in the Compute lab and it is meant to last the whole semester. Use that same jumpbox here. The Azure CLI runs there, not on your laptop.

No jumpbox yet? Set one up using the “Set up your lab environment” section of the Compute lab README (an Ubuntu 24.04 LTS VM in its own `lab-jumpbox-rg`, with the Azure CLI installed and `az login` done). Then come back here. Everything below assumes that jumpbox exists and you are logged in with `az`.

Region tip. Azure capacity and feature availability vary by region. The default is `centralus`. If a create fails with a capacity or availability error, change `LAB_LOCATION` in `lab-config.sh` and try another region such as `eastus2`, `westus2`, `westus3`, `southcentralus`, `westeurope`, or `northeurope`.

Getting started on your jumpbox

1. **Connect to the jumpbox.** The easiest way is **VS Code Remote-SSH**: you get a file explorer, a rendered preview of each `problem.md` and `solution.md`, and a terminal, all running on the jumpbox. Plain `ssh <user>@<public-ip>` or the Portal's Bastion work too. (See the Compute lab README if you need the Remote-SSH walkthrough again.)

2. **Clone this repo onto the jumpbox:**

```
git clone https://github.com/sailplatform/cloud-admin-lab-exercises-storage.git
cd cloud-admin-lab-exercises-storage
```

3. **Do the two one time prep steps for the SQL questions** (the Blob questions need nothing extra):

```
# a) Register the Azure SQL resource provider on your subscription (one time).
# The SQL setups also do this, but running it now avoids a first run surprise.
az provider register --namespace Microsoft.Sql --wait

# b) Get a SQL client. Azure Cloud Shell has sqlcmd preinstalled. To install it on
# the Ubuntu jumpbox, use Microsoft's mssql-tools18 package:
curl -sSL -O https://packages.microsoft.com/config/ubuntu/$(lsb_release
↵ -rs)/packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
sudo apt-get update
```

```
sudo ACCEPT_EULA=Y apt-get install -y mssql-tools18 unixodbc-dev
echo 'export PATH="$PATH:/opt/mssql-tools18/bin"' >> ~/.bashrc && source ~/.bashrc
```

4. Open the repo in VS Code and start. Use **File, Open Folder**, cloud-admin-lab-exercises-storage. Open a question's problem.md, hit the preview icon to read it rendered, then run the scripts in the terminal:

```
cd 1-blob-storage/Question-1-Create-Storage-Account
./setup.bash          # then read problem.md and solve it
```

Cost warning. These labs create **real Azure resources that cost real money**. Each question is isolated in its own resource group and uses cheap SKUs. You are billed while resources exist, so the golden rule is: provision, solve, validate, then **run ./cleanup.bash the moment you finish a question**. The General Purpose database in the serverless question costs more than the others, so be especially prompt cleaning that one up.

Keep your jumpbox for the whole semester; do not delete it. Just **stop (deallocate)** it when you are done for the day so it stops charging for compute, and **Start** it again next time. Only delete lab-jumpbox-rg at the end of the course.

How each question is structured

Every graded question lives in its own self contained folder with five files:

File	What it is
setup.bash	Prepares or pre checks the environment (and, where relevant, provisions the starting resource).
problem.md	The scenario and the exact requirements ("done" criteria).
validate.bash	Automated PASS or FAIL checks against your subscription.
solution.md	A concise walkthrough of how an admin reasons about it. Read this if you get stuck.
cleanup.bash	Deletes everything the question created.

setup.bash, validate.bash, and cleanup.bash are executable scripts you run from inside the question folder. No central runner, no question numbers to remember. All scripts are open source: read them.

The workflow

Everything happens inside a single question folder. cd into it and go:

```
cd 1-blob-storage/Question-1-Create-Storage-Account

./setup.bash          # 1. Prepare or preflight (and, where relevant, provision a starting resource)

# 2. Read problem.md and solve it yourself with the Azure CLI.
#   Stuck? Open solution.md in the same folder.

./validate.bash       # 3. Check your work. Prints PASS or FAIL for each requirement.

./cleanup.bash        # 4. Delete the resources so they stop costing money.
```

Configuration: make it yours

Shared settings live in [lab-config.sh](#) at the repo root: the **region** (LAB_LOCATION, default `centralus`), the resource group prefix (LAB_RG_PREFIX, default `lab-storage`), and the SQL admin login name (LAB_SQL_ADMIN, default `sqladmin`; the password is never stored, you set it yourself). Change a value once and every script follows, or override for a single run:

```
LAB_LOCATION=westus3 ./validate.bash
```

A note on passwords. For the SQL questions you choose the server's admin password. It is your secret: never commit it to a file or share it. The lab only ever stores the non secret login name.

Available questions

The lab is organized by category. Work each question from inside its folder.

1-blob-storage/: Blob Storage (object storage)

#	Topic	Skill
1	Create a Storage Account and Container	Create an account (kind and SKU) and a blob container
2	Upload a Blob	Put a file into a container with the right data plane auth
3	Block Public Access	Prove a blob is public with curl, then make it private
4	Optimize Cost with the Access Tier	Move an account default tier from Hot to Cool

2-azure-sql/: Azure SQL Database

#	Topic	Skill
1	Create a Logical Server and a Database	Create a server (you set the password) and a Basic database
2	Configure the Server Firewall	Connect with sqlcmd, see it refused, open your IP, connect again
3	Scale the Database	Change the performance level from Basic to Standard S0
4	Serverless with Auto Pause	Convert to a serverless model that pauses compute when idle

Plus [ten practice exercises](#) (five Blob, five SQL: redundancy, SAS tokens, soft delete, lifecycle policies, tables and data, point in time restore, and a capstone that exports a database to Blob Storage).

The practice mindset (why solutions are given freely)

First study the worked examples, the graded questions, each with a `solution.md` walkthrough of how an admin reasons through the task. Then drill the practice set (problems only, no solutions) to build muscle memory. The point is not to know how to create a storage account or a database. It is to prove you can, and to build the habit of finding the answer when you do not already have it.

Part 1 — Lab Questions

Question 1 — Create a Storage Account and Container

Scenario

The team needs a place in Azure to store files — documents, backups, exports. As the cloud administrator, create a **storage account** and a **blob container** to hold them.

Requirements

#	Requirement	Value
1	A resource group	lab-storage-blob01-rg, in your lab region
2	A storage account (globally-unique name)	kind StorageV2, SKU Standard_LRS
3	A blob container	named data

Storage-account names are **globally unique** and must be all lowercase letters and numbers (3–24 chars) — pick something like labstore<yourinitials><digits>. The validator finds the account in your resource group, so the exact name is your choice.

Work the question

```
./setup.bash      # checks your environment and prints the target
# ... create the resources yourself (see solution.md if you get stuck) ...
./validate.bash   # PASS/FAIL check
./cleanup.bash    # delete everything when you're done
```

Question 2 — Upload a File to Blob Storage

Scenario

The storage account and its **data** container already exist — `setup.bash` provisioned them. Now put a file in it: as the cloud administrator, **upload a file to the data container** so the team can share it.

Requirements

#	Requirement	Value
1	A storage account + data container	in <code>lab-storage-blob02-rg</code> (setup did this)
2	A blob exists in the data container	named <code>hello.txt</code>

The storage account name was generated by setup. Find it with `az storage account list -g lab-storage-blob02-rg --query "[0].name" -o tsv`.

Work the question

```
./setup.bash      # provisions the account + container, prints the target
# ... upload a file yourself (see solution.md if you get stuck) ...
./validate.bash
./cleanup.bash
```

Question 3 — Lock Down the Storage Account

Scenario

A security review flagged a storage account: a container is set to **anonymous public access**, so anyone with the blob's URL can read it — no credentials required. As the cloud administrator, prove the exposure, then harden the account so the data can't be read without authentication.

`setup.bash` provisions the account in this insecure state and uploads a file (`secret.txt`) into a public container so there is something real to test. Before you harden anything, `curl` the blob URL it prints and watch it return the file — then harden, and watch the same URL start refusing you. (Walkthrough in `solution.md`.)

Requirements

The storage account in `lab-storage-blob03-rg` must have:

#	Requirement	Value
1	Public blob access disabled	<code>allowBlobPublicAccess = false</code>

Work the question

```
./setup.bash          # provisions the exposed account, prints the blob URL
# ... confirm it's public, harden it, confirm it's blocked (see solution.md) ...
./validate.bash
./cleanup.bash
```

Question 4 — Optimize Cost with the Access Tier

Scenario

A storage account holds data that is **rarely read** — old reports kept only for the occasional audit. It sits on the **Hot** tier, which charges a premium to keep data instantly ready. For infrequently-accessed data, the **Cool** tier costs less to store (in exchange for higher per-read charges and a 30-day minimum retention). As the cloud administrator, move the account's default access tier to **Cool**.

`setup.bash` provisions the account on Hot and uploads a file whose tier is *inferred* from the account default. Check the account and the blob before and after your change: because the blob has no tier of its own, it follows the account default — set the account to Cool and watch the blob move with it. (Walkthrough in `solution.md`.)

Requirements

The storage account in `lab-storage-blob04-rg` must have:

#	Requirement	Value
1	Account default access tier	<code>accessTier = Cool</code>

Work the question

```
./setup.bash          # provisions the Hot account + blob, prints the check commands
# ... see Hot, switch to Cool, see Cool ...
./validate.bash
./cleanup.bash
```

Question 1 — Create a Logical Server and a Database

Scenario

The team needs a managed SQL database in Azure for a small app. In Azure SQL, a **database always lives inside a logical server** — the server is the connection endpoint (`<name>.database.windows.net`) and holds the admin login; the database holds your data. As the cloud administrator, create both, using the cheap **Basic** tier.

Requirements

#	Requirement	Value
1	A resource group	lab-storage-sql01-rg, in your lab region
2	A SQL logical server (globally-unique name)	admin login sqladmin; you choose the password
3	A database on the server	name appdb, tier Basic

*Server names are globally unique (they become a public DNS name), all lowercase — pick e.g. labsql<initials><digits>. The validator finds the server in your resource group, so the exact name is your choice. **The admin password is a secret only you set** — never put it in a script or share it.*

Once it's built, confirm the database reports **Online** on the **Basic** tier (walkthrough in `solution.md`). You'll notice you still can't *connect* a SQL client to it — the server firewall blocks all IPs by default. Opening that is **Question 2**.

Work the question

```
./setup.bash          # preflight only - you build everything
# ... create the RG, server (your password), and appdb ...
./validate.bash
./cleanup.bash
```

Question 2 — Configure the Server Firewall

Scenario

The `appdb` database from Q1 is running, but nobody can connect to it: an Azure SQL logical server **blocks every IP address by default**. A teammate needs to reach it from their workstation. As the cloud administrator, open the server firewall for a client IP — then prove a client can now connect.

`setup.bash` provisions the server and `appdb` for you (you set the password). Prove it end-to-end with a SQL client (`sqlcmd`): try to connect *before* adding a rule and the gateway refuses you by IP; add the rule; run the same command and it returns a row. (Walkthrough — including where to get `sqlcmd` — in `solution.md`.)

Requirements

The SQL server in `lab-storage-sql02-rg` must have:

#	Requirement	Value
1	A firewall rule allowing your client's public IP	start/end IP cover your current IP

Allow ***your specific IP***, not `0.0.0.0-255.255.255.255` (that opens the server to the whole internet). The validator checks that some rule covers the IP you're running from.

Work the question

```
./setup.bash          # provisions server + appdb (you set the password)
# ... try sqlcmd (blocked) -> add rule -> sqlcmd (works) ...
./validate.bash
./cleanup.bash
```

Question 3 — Scale the Database

Scenario

appdb is on the **Basic** tier (5 DTU) and the app has outgrown it — queries are throttling under load. Azure SQL lets you change a database's performance level **online**, without recreating it. As the cloud administrator, scale appdb up to the **Standard S0** performance level (10 DTU).

setup.bash provisions the server and a Basic appdb for you.

First, two terms: DTU and service objective

- **DTU = Database Transaction Unit.** It's Azure's single, blended unit of database power — a bundle of **CPU + memory + I/O (reads/writes)** rolled into one number. You don't size those three separately; you just pick how many DTUs you want, and more DTUs means proportionally more of all three. (This is the "DTU purchasing model." Azure also has a "vCore" model where you choose CPU cores and memory directly — that's a later topic.)
- **Service objective** (a.k.a. *performance level*) is the named tier that sets the DTU count: **Basic** (5 DTU), **S0** (10 DTU), **S1** (20 DTU), and up. **Scaling a DTU database is just changing this one value.**

Check the database's level before and after: it starts at Basic / 5 DTU, and after you scale it reads Standard / S0 / 10 DTU — while staying **Online** the whole time (no downtime, no data lost). (Walkthrough in solution.md.)

Requirements

#	Requirement	Value
1	appdb performance level	currentServiceObjectiveName = S0 (Standard edition)

Work the question

```
./setup.bash          # provisions server + Basic appdb (you set the password)
# ... see Basic/5 -> scale to S0 -> see Standard/S0/10 ...
./validate.bash
./cleanup.bash
```

Question 4 — Serverless with Auto-Pause

Scenario

appdb is a **provisioned** General Purpose database: its compute is allocated around the clock, so you pay for it 24/7 even when no one is querying — like leaving a VM running idle. For a database used only occasionally (a dev/test DB, an internal tool), the **Serverless** compute model is cheaper: it bills compute per-second, **auto-pauses** after an idle period (compute billing drops to zero, you keep paying only for storage), and **auto-resumes** on the next connection.

As the cloud administrator, convert appdb to Serverless with a 60-minute auto-pause delay. This is the database analogue of *deallocating* an idle VM.

setup.bash provisions appdb as a provisioned database. Check its settings before and after: a provisioned database has no auto-pause fields, and after you convert it the SKU gains an `_S_` (serverless) and the auto-pause settings appear. The pause *itself* happens only after 60 minutes idle (Azure's minimum), so you won't watch it flip during the lab — but from then on `az sql db show --query status` reports `Paused` after an idle hour, and the next query resumes it in seconds. (Walkthrough in `solution.md`.)

Requirements

The appdb database in lab-storage-sql04-rg must be:

#	Requirement	Value
1	Compute model	Serverless (SKU like GP_S_Gen5_1)
2	Auto-pause delay	<code>autoPauseDelay</code> = 60 minutes

Work the question

```
./setup.bash      # provisions server + provisioned appdb (you set the password)
# ... see no auto-pause -> convert to serverless -> see auto-pause settings ...
./validate.bash
./cleanup.bash    # do this promptly - General Purpose isn't free
```

Part 2 — Solutions

Question 1 — Create a Storage Account and Container

Scenario

The team needs a place in Azure to store files — documents, backups, exports. As the cloud administrator, create a **storage account** and a **blob container** to hold them.

Requirements

#	Requirement	Value
1	A resource group	lab-storage-blob01-rg, in your lab region
2	A storage account (globally-unique name)	kind StorageV2, SKU Standard_LRS
3	A blob container	named data

Storage-account names are **globally unique** and must be all lowercase letters and numbers (3–24 chars) — pick something like labstore<yourinitials><digits>. The validator finds the account in your resource group, so the exact name is your choice.

Work the question

```
./setup.bash      # checks your environment and prints the target
# ... create the resources yourself (see solution.md if you get stuck) ...
./validate.bash   # PASS/FAIL check
./cleanup.bash    # delete everything when you're done
```

Solution — Create a Storage Account and Container

Start at the reference. Every az command is documented — for this one it's *Azure CLI reference* → *Storage* → *az storage account* → *create*: <https://learn.microsoft.com/cli/azure/storage/account#az-storage-account-create>. Read it (or run `az storage account create --help`): it explains every parameter, lists the **allowed values**, and — importantly — shows each flag's **default**. A flag you don't pass takes its default, so only pass one when you want something *other* than the default.

What you're actually choosing:

- **--kind (account kind)** — its default is already **StorageV2**, the modern general-purpose kind (blobs, files, queues, tables + access tiering). So you **don't pass --kind at all** — a plain create already gives you StorageV2. (The docs list the alternatives you *would* name explicitly: **BlobStorage**, **FileStorage**, **BlockBlobStorage**, **StorageV1**.)

- **--sku (redundancy)** — how many copies Azure keeps, and where. Here the default is **Standard_RAGRS** (geo-redundant across two regions, ~2× the cost), so we override it: **--sku Standard_LRS = Locally Redundant Storage**, 3 copies in one datacenter, the cheapest — plenty for a lab. That’s the rule: keep defaults, override *deliberately*.

A **storage account** is the top-level container for all of Azure Storage; its name is a global DNS label (unique, all-lowercase). Your files (**blobs**) live inside **containers**.

```
az group create -n lab-storage-blob01-rg -l centralus

az storage account create -g lab-storage-blob01-rg -n <unique-name> --sku Standard_LRS

az storage container create --account-name <unique-name> --name data --auth-mode key
```

Auth note: container/blob commands use **--auth-mode key** (the account key, which an Owner/Contributor can fetch automatically). **--auth-mode login** uses your Entra identity instead — but that needs the *Storage Blob Data Contributor* role; being subscription Owner isn’t enough for data-plane access.

Verify: az storage container exists --account-name <unique-name> --name data --auth-mode key

Goal	Command
See flags, allowed values, defaults	az storage account create --help
Create account (LRS; kind defaults to StorageV2)	az storage account create -g <rg> -n <acct> --sku Standard_LRS
List accounts in an RG	az storage account list -g <rg> -o table
Create container	az storage container create --account-name <acct> -n <name> --auth-mode key

Question 2 — Upload a File to Blob Storage

Scenario

The storage account and its **data** container already exist — `setup.bash` provisioned them. Now put a file in it: as the cloud administrator, **upload a file to the data container** so the team can share it.

Requirements

#	Requirement	Value
1	A storage account + data container	in <code>lab-storage-blob02-rg</code> (setup did this)
2	A blob exists in the data container	named <code>hello.txt</code>

The storage account name was generated by setup. Find it with `az storage account list -g lab-storage-blob02-rg --query "[0].name" -o tsv`.

Work the question

```
./setup.bash      # provisions the account + container, prints the target
# ... upload a file yourself (see solution.md if you get stuck) ...
./validate.bash
./cleanup.bash
```

Solution — Upload a File to Blob Storage

Reference: *Azure CLI reference* → *Storage* → *az storage blob* → *upload*: <https://learn.microsoft.com/cli/azure/storage/blob#az-storage-blob-upload> (or `az storage blob upload --help`).

A **blob** is just a file stored inside a container. `az storage blob upload` copies a **local file** (`--file`) into a container (`--container-name`) under a blob name (`--name`). So you need a file on disk first, then the account name (setup generated it — look it up rather than guessing).

```
# 1) Make a file to upload (any file works)
echo "Hello, Azure Storage" > hello.txt

# 2) Find the account setup created
ACCT=$(az storage account list -g lab-storage-blob02-rg --query "[0].name" -o tsv)

# 3) Upload it as a blob named hello.txt
az storage blob upload --account-name "$ACCT" --container-name data \
  --name hello.txt --file hello.txt --auth-mode key
```

Verify: `az storage blob list --account-name "$ACCT" --container-name data --auth-mode key -o table`

Goal	Command
See upload flags	<code>az storage blob upload --help</code>
Upload a blob	<code>az storage blob upload --account-name <acct> --container-name <c> --name <blob> -file <local> --auth-mode key</code>

Goal	Command
List blobs	<pre>az storage blob list --account-name <acct> --container-name <c> --auth-mode key -o table</pre>
Download a blob	<pre>az storage blob download --account-name <acct> --container-name <c> --name <blob> - -file <local> --auth-mode key</pre>

Question 3 — Lock Down the Storage Account

Scenario

A security review flagged a storage account: a container is set to **anonymous public access**, so anyone with the blob's URL can read it — no credentials required. As the cloud administrator, prove the exposure, then harden the account so the data can't be read without authentication.

`setup.bash` provisions the account in this insecure state and uploads a file (`secret.txt`) into a public container so there is something real to test. Before you harden anything, `curl` the blob URL it prints and watch it return the file — then harden, and watch the same URL start refusing you. (Walkthrough in `solution.md`.)

Requirements

The storage account in `lab-storage-blob03-rg` must have:

#	Requirement	Value
1	Public blob access disabled	<code>allowBlobPublicAccess = false</code>

Work the question

```
./setup.bash      # provisions the exposed account, prints the blob URL
# ... confirm it's public, harden it, confirm it's blocked (see solution.md) ...
./validate.bash
./cleanup.bash
```

Solution — Lock Down the Storage Account

Reference: *Azure CLI reference* → *Storage* → *az storage account* → *update*: <https://learn.microsoft.com/cli/azure/storage/account#az-storage-account-update> (or `az storage account update --help`). `update` changes settings on an existing account — same flags as `create`.

Move 1 — see the exposure

Get the blob's URL (setup printed it, but here's how to find it yourself), then request it with no credentials:

```
ACCT=$(az storage account list -g lab-storage-blob03-rg --query "[0].name" -o tsv)

URL=$(az storage blob url --account-name "$ACCT" \
    --container-name public-data --name secret.txt --auth-mode key -o tsv)

curl -i "$URL"      # -> HTTP/1.1 200 OK, and the file contents. Anyone could read this.
```

The URL is just `https://<account>.blob.core.windows.net/<container>/<blob>` — a plain public web address. That is why an exposed container is dangerous.

Move 2 — harden the account

What you're changing:

- `--allow-blob-public-access false` — turns off *anonymous* read access for the whole account. This account-level switch overrides the container's own `public-access` setting, so it shuts the door even though the container is still marked `blob`. New accounts default to `false`; setup turned it on.

```
az storage account update -g lab-storage-blob03-rg -n "$ACCT" \
  --allow-blob-public-access false
```

Move 3 — confirm it's closed

```
curl -i "$URL"      # -> HTTP/1.1 409, <Error><Code>PublicAccessNotPermitted</Code>...
```

The same URL that served the file a minute ago now refuses it. Disabling public access *at the account level* returns **409 PublicAccessNotPermitted** (a container-level or other auth refusal would be 403) — either way, the anonymous request is blocked.

Verify the setting: `az storage account show -g lab-storage-blob03-rg -n "$ACCT" --query "{public:allowBlobPublicAccess}" -o table`

Goal	Command
Get a blob's public URL	<code>az storage blob url --account-name <acct> - -container-name <c> --name --auth-mode key -o tsv</code>
Test anonymous access	<code>curl -i "<url>"</code> (200 = exposed; 403/409 = blocked)
See update flags	<code>az storage account update --help</code>
Block public blob access	<code>az storage account update -g <rg> -n <acct> --allow-blob-public-access false</code>
Check current setting	<code>az storage account show -g <rg> -n <acct> - -query "{public:allowBlobPublicAccess}" -o table</code>

Question 4 — Optimize Cost with the Access Tier

Scenario

A storage account holds data that is **rarely read** — old reports kept only for the occasional audit. It sits on the **Hot** tier, which charges a premium to keep data instantly ready. For infrequently-accessed data, the **Cool** tier costs less to store (in exchange for higher per-read charges and a 30-day minimum retention). As the cloud administrator, move the account’s default access tier to **Cool**.

`setup.bash` provisions the account on Hot and uploads a file whose tier is *inferred* from the account default. Check the account and the blob before and after your change: because the blob has no tier of its own, it follows the account default — set the account to Cool and watch the blob move with it. (Walkthrough in `solution.md`.)

Requirements

The storage account in `lab-storage-blob04-rg` must have:

#	Requirement	Value
1	Account default access tier	<code>accessTier = Cool</code>

Work the question

```
./setup.bash      # provisions the Hot account + blob, prints the check commands
# ... see Hot, switch to Cool, see Cool ...
./validate.bash
./cleanup.bash
```

Solution — Optimize Cost with the Access Tier

Reference: *Azure CLI reference* → *Storage* → *az storage account* → *update*: <https://learn.microsoft.com/cli/azure/storage/account#az-storage-account-update> (or `az storage account update --help`). Look at `--access-tier` — allowed values Cold, Cool, Hot, Premium.

What the access tier is

The **access tier** decides how you’re billed for blob storage — it’s a cost/performance trade-off, not a capability switch:

- **Hot** — highest storage cost, lowest access cost. For data read often.
- **Cool** — lower storage cost, higher access cost, 30-day minimum retention. For data read rarely (backups, old reports).
- **Cold/Premium** exist too; Cool is the usual “save money on cold-ish data” pick.

The account has a **default** tier. A blob can carry its own explicit tier, but a blob uploaded without one has an *inferred* tier that simply follows the account default — which is why switching the account moves that blob too.

Move 1 — see the starting state (Hot)

```
ACCT=$(az storage account list -g lab-storage-blob04-rg --query "[0].name" -o tsv)
az storage account show -g lab-storage-blob04-rg -n "$ACCT" --query accessTier -o tsv
```

```
az storage blob show --account-name "$ACCT" --container-name data \
  --name report.csv --auth-mode key --query properties.blobTier -o tsv
# both print: Hot
```

Move 2 — switch the account default to Cool

```
az storage account update -g lab-storage-blob04-rg -n "$ACCT" --access-tier Cool
```

Move 3 — confirm (Cool)

```
az storage account show -g lab-storage-blob04-rg -n "$ACCT" --query accessTier -o tsv
az storage blob show --account-name "$ACCT" --container-name data \
  --name report.csv --auth-mode key --query properties.blobTier -o tsv
# both now print: Cool
```

Billing changes aren't instantly visible on your invoice, but the tier that *drives* the bill is — and you watched the untagged blob inherit it.

Goal	Command
See account default tier	az storage account show -g <rg> -n <acct> - -query accessTier -o tsv
See a blob's effective tier	az storage blob show --account-name <acct> --container-name <c> --name --auth-mode key --query properties.blobTier -o tsv
See allowed tier values	az storage account update --help
Set account default to Cool	az storage account update -g <rg> -n <acct> --access-tier Cool
Set a <i>single blob's</i> tier	az storage blob set-tier --account-name <acct> --container-name <c> --name -- tier Cool --auth-mode key

Question 1 — Create a Logical Server and a Database

Scenario

The team needs a managed SQL database in Azure for a small app. In Azure SQL, a **database always lives inside a logical server** — the server is the connection endpoint (`<name>.database.windows.net`) and holds the admin login; the database holds your data. As the cloud administrator, create both, using the cheap **Basic** tier.

Requirements

#	Requirement	Value
1	A resource group	lab-storage-sql01-rg, in your lab region
2	A SQL logical server (globally-unique name)	admin login sqladmin; you choose the password
3	A database on the server	name appdb, tier Basic

*Server names are globally unique (they become a public DNS name), all lowercase — pick e.g. labsql<initials><digits>. The validator finds the server in your resource group, so the exact name is your choice. **The admin password is a secret only you set** — never put it in a script or share it.*

Once it's built, confirm the database reports **Online** on the **Basic** tier (walkthrough in `solution.md`). You'll notice you still can't *connect* a SQL client to it — the server firewall blocks all IPs by default. Opening that is **Question 2**.

Work the question

```
./setup.bash          # preflight only - you build everything
# ... create the RG, server (your password), and appdb ...
./validate.bash
./cleanup.bash
```

Solution — Create a Logical Server and a Database

Reference: *Azure CLI reference* → *SQL*:

- server: <https://learn.microsoft.com/cli/azure/sql/server#az-sql-server-create>
- database: <https://learn.microsoft.com/cli/azure/sql/db#az-sql-db-create>

(or `az sql server create --help` / `az sql db create --help`).

Two resources, in order

1. **Logical server** — the endpoint + admin identity. `-u` is the admin login; `-p` is its password. Azure requires a strong password (`>=8` chars, 3 of: upper, lower, digit, symbol) — **you** choose it and replace the placeholder below. Don't save a real password into a file or commit it to a repo.

```
az group create -n lab-storage-sql01-rg -l centralus

az sql server create -g lab-storage-sql01-rg -n <unique-server> \
  -u sqladmin -p '<your-password>'
```

2. **Database** — created *inside* the server with `-s`. The tier is set with `-e/--edition`. Run `az sql db create --help`: the default edition is **General Purpose** (several vCores, pricey), so pass **Basic** deliberately — it's the cheapest tier (~2 GB, 5 DTU), perfect for a lab.

```
az sql db create -g lab-storage-sql01-rg -s <unique-server> -n appdb -e Basic
```

Confirm it's running

```
az sql db show -g lab-storage-sql01-rg -s <unique-server> -n appdb \
  --query "{name:name, tier:edition, status:status}" -o table
# name tier status
# appdb Basic Online
```

The database is live — but a SQL client on your laptop still can't reach it until you open the server firewall (Question 2).

Goal	Command
See server flags	<code>az sql server create --help</code>
See db tiers/defaults	<code>az sql db create --help</code>
Create server	<code>az sql server create -g <rg> -n <server> -u sqladmin -p '<your-password>'</code>
Create Basic database	<code>az sql db create -g <rg> -s <server> -n appdb -e Basic</code>
List databases on a server	<code>az sql db list -g <rg> -s <server> -o table</code>
Check one database	<code>az sql db show -g <rg> -s <server> -n appdb --query "{tier:edition, status:status}" -o table</code>

Question 2 — Configure the Server Firewall

Scenario

The `appdb` database from Q1 is running, but nobody can connect to it: an Azure SQL logical server **blocks every IP address by default**. A teammate needs to reach it from their workstation. As the cloud administrator, open the server firewall for a client IP — then prove a client can now connect.

`setup.bash` provisions the server and `appdb` for you (you set the password). Prove it end-to-end with a SQL client (`sqlcmd`): try to connect *before* adding a rule and the gateway refuses you by IP; add the rule; run the same command and it returns a row. (Walkthrough — including where to get `sqlcmd` — in `solution.md`.)

Requirements

The SQL server in `lab-storage-sql02-rg` must have:

#	Requirement	Value
1	A firewall rule allowing your client's public IP	start/end IP cover your current IP

Allow ***your specific IP***, not `0.0.0.0-255.255.255.255` (that opens the server to the whole internet). The validator checks that some rule covers the IP you're running from.

Work the question

```
./setup.bash      # provisions server + appdb (you set the password)
# ... try sqlcmd (blocked) -> add rule -> sqlcmd (works) ...
./validate.bash
./cleanup.bash
```

Solution — Configure the Server Firewall

Reference: *Azure CLI reference* → *SQL* → *az sql server firewall-rule*: <https://learn.microsoft.com/cli/azure/sql/server/firewall-rule#az-sql-server-firewall-rule-create> (or `az sql server firewall-rule create --help`).

The server firewall is an **allow-list of IP ranges** at the server (not the database) level. Empty list = nobody connects. You add your client's public IP.

Move 1 — see it blocked

```
SRV=$(az sql server list -g lab-storage-sql02-rg --query "[0].name" -o tsv)
FQDN=$(az sql server show -g lab-storage-sql02-rg -n "$SRV" \
  --query fullyQualifiedDomainName -o tsv)

sqlcmd -S "$FQDN" -U sqladmin -P '<your-password>' -d appdb -Q "SELECT 'connected' AS status"
# -> Cannot open server ... Client with IP address 'x.x.x.x' is not allowed...
```

No `sqlcmd`? Easiest is **Azure Cloud Shell** (it's preinstalled — just add a firewall rule for Cloud Shell's egress IP the same way). To install it on the Ubuntu jumpbox, use Microsoft's `mssql-tools18` package (the `sqlcmd`/`bcp` tools). Note the package is `mssql-tools18`, *not* `sqlcmd`, it needs the EULA accepted, and it installs under `/opt/mssql-tools18/bin`:

```
curl -sSL -O https://packages.microsoft.com/config/ubuntu/$(lsb_release
↪ -rs)/packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
sudo apt-get update
sudo ACCEPT_EULA=Y apt-get install -y mssql-tools18 unixodbc-dev
echo 'export PATH="$PATH:/opt/mssql-tools18/bin"' >> ~/.bashrc && source ~/.bashrc
```

Move 2 — allow your IP

Find your public IP and open a rule for exactly that address:

```
MYIP=$(curl -s -4 https://api.ipify.org)           # your IPv4 (firewall is IPv4-only)

az sql server firewall-rule create -g lab-storage-sql02-rg -s "$SRV" \
  -n allow-my-client --start-ip-address "$MYIP" --end-ip-address "$MYIP"
```

Use your **one** IP for start and end. A range like 0.0.0.0–255.255.255.255 would “work” but exposes the server to the entire internet — don’t.

Move 3 — confirm

```
sqlcmd -S "$FQDN" -U sqladmin -P '<your-password>' -d appdb -Q "SELECT 'connected' AS status"
# status
# -----
# connected
```

The identical command that was refused now returns a row. (Firewall changes take effect within a few seconds.)

Goal	Command
Get your public IPv4	<code>curl -s -4 https://api.ipify.org</code>
See firewall-rule flags	<code>az sql server firewall-rule create --help</code>
Allow your IP	<code>az sql server firewall-rule create -g <rg> -s <server> -n allow-my-client --start-ip-address <ip> --end-ip-address <ip></code>
List firewall rules	<code>az sql server firewall-rule list -g <rg> -s <server> -o table</code>
Server endpoint (FQDN)	<code>az sql server show -g <rg> -n <server> --query fullyQualifiedDomainName -o tsv</code>

Question 3 — Scale the Database

Scenario

appdb is on the **Basic** tier (5 DTU) and the app has outgrown it — queries are throttling under load. Azure SQL lets you change a database's performance level **online**, without recreating it. As the cloud administrator, scale appdb up to the **Standard S0** performance level (10 DTU).

setup.bash provisions the server and a Basic appdb for you.

First, two terms: DTU and service objective

- **DTU = Database Transaction Unit.** It's Azure's single, blended unit of database power — a bundle of **CPU + memory + I/O (reads/writes)** rolled into one number. You don't size those three separately; you just pick how many DTUs you want, and more DTUs means proportionally more of all three. (This is the "DTU purchasing model." Azure also has a "vCore" model where you choose CPU cores and memory directly — that's a later topic.)
- **Service objective** (a.k.a. *performance level*) is the named tier that sets the DTU count: **Basic** (5 DTU), **S0** (10 DTU), **S1** (20 DTU), and up. **Scaling a DTU database is just changing this one value.**

Check the database's level before and after: it starts at Basic / 5 DTU, and after you scale it reads Standard / S0 / 10 DTU — while staying **Online** the whole time (no downtime, no data lost). (Walkthrough in solution.md.)

Requirements

#	Requirement	Value
1	appdb performance level	currentServiceObjectiveName = S0 (Standard edition)

Work the question

```
./setup.bash          # provisions server + Basic appdb (you set the password)
# ... see Basic/5 -> scale to S0 -> see Standard/S0/10 ...
./validate.bash
./cleanup.bash
```

Solution — Scale the Database

Reference: Azure CLI reference → SQL → az sql db update: <https://learn.microsoft.com/cli/azure/sql/db#az-sql-db-update> (or az sql db update --help). update changes an existing database in place.

Move 1 — see the current level

```
SRV=$(az sql server list -g lab-storage-sql03-rg --query "[0].name" -o tsv)

az sql db show -g lab-storage-sql03-rg -s "$SRV" -n appdb \
  --query "{tier:edition, objective:currentServiceObjectiveName, dtu:sku.capacity}" -o table
# Basic / Basic / 5
```

Move 2 — scale to S0

The performance level is the **service objective**. S0 is a standalone objective name that implies Standard edition, so one flag does it:

```
az sql db update -g lab-storage-sql03-rg -s "$SRV" -n appdb --service-objective S0
```

(Equivalent, spelled out: `--edition Standard --capacity 10`. Run `az sql db update --help` to see `--service-objective`, `--edition`, and `--capacity`.)

Move 3 — confirm

```
az sql db show -g lab-storage-sql03-rg -s "$SRV" -n appdb \  
  --query "{tier:edition, objective:currentServiceObjectiveName, dtu:sku.capacity}" -o table  
# Standard / S0 / 10
```

The edition, objective, and DTU count all moved up, and **status** stayed **Online** throughout — scaling a DTU database is an online operation. To scale back down you'd run the same command with `--service-objective Basic`.

Goal	Command
See current level	<pre>az sql db show -g <rg> -s <server> -n appdb --query "{tier:edition, objective: currentServiceObjectiveName, dtu:sku. capacity}" -o table</pre>
See update flags	<pre>az sql db update --help</pre>
Scale to S0	<pre>az sql db update -g <rg> -s <server> -n appdb --service-objective S0</pre>
Scale back to Basic	<pre>az sql db update -g <rg> -s <server> -n appdb --service-objective Basic</pre>

Question 4 — Serverless with Auto-Pause

Scenario

appdb is a **provisioned** General Purpose database: its compute is allocated around the clock, so you pay for it 24/7 even when no one is querying — like leaving a VM running idle. For a database used only occasionally (a dev/test DB, an internal tool), the **Serverless** compute model is cheaper: it bills compute per-second, **auto-pauses** after an idle period (compute billing drops to zero, you keep paying only for storage), and **auto-resumes** on the next connection.

As the cloud administrator, convert appdb to Serverless with a 60-minute auto-pause delay. This is the database analogue of *deallocating* an idle VM.

setup.bash provisions appdb as a provisioned database. Check its settings before and after: a provisioned database has no auto-pause fields, and after you convert it the SKU gains an `_S_` (serverless) and the auto-pause settings appear. The pause *itself* happens only after 60 minutes idle (Azure's minimum), so you won't watch it flip during the lab — but from then on `az sql db show --query status` reports `Paused` after an idle hour, and the next query resumes it in seconds. (Walkthrough in `solution.md`.)

Requirements

The appdb database in lab-storage-sql04-rg must be:

#	Requirement	Value
1	Compute model	Serverless (SKU like GP_S_Gen5_1)
2	Auto-pause delay	<code>autoPauseDelay = 60 minutes</code>

Work the question

```
./setup.bash      # provisions server + provisioned appdb (you set the password)
# ... see no auto-pause -> convert to serverless -> see auto-pause settings ...
./validate.bash
./cleanup.bash    # do this promptly - General Purpose isn't free
```

Solution — Serverless with Auto-Pause

Reference: Azure CLI reference → SQL → `az sql db update`: <https://learn.microsoft.com/cli/azure/sql/db#az-sql-db-update> (or `az sql db update --help`). The serverless flags are `--compute-model`, `--auto-pause-delay`, and `--min-capacity`.

What “serverless” changes

Serverless is a **compute model** of the vCore General Purpose tier, not a separate edition. Instead of a fixed vCore count billed 24/7, you set a **range** — `--min-capacity` (floor while active) up to `--capacity` (ceiling) — and Azure scales compute within it and **auto-pauses** after `--auto-pause-delay` idle minutes. Paused = you pay only for storage.

Move 1 — see the starting state

```
SRV=$(az sql server list -g lab-storage-sql04-rg --query "[0].name" -o tsv)
az sql db show -g lab-storage-sql04-rg -s "$SRV" -n appdb \
```

```
--query "{sku:currentServiceObjectiveName, autoPause:autoPauseDelay, minCap:minCapacity}" -o
↪ table
# GP_Gen5_2 / (blank) / (blank)
```

Move 2 — convert to serverless

Keep General Purpose, switch the compute model, and set the auto-pause window. The minimum delay Azure allows is **60** minutes (use -1 to disable auto-pause):

```
az sql db update -g lab-storage-sql04-rg -s "$SRV" -n appdb \
  --compute-model Serverless -e GeneralPurpose -f Gen5 \
  --capacity 1 --min-capacity 0.5 --auto-pause-delay 60
```

Move 3 — confirm

```
az sql db show -g lab-storage-sql04-rg -s "$SRV" -n appdb \
  --query "{sku:currentServiceObjectiveName, autoPause:autoPauseDelay, minCap:minCapacity}" -o
↪ table
# GP_S_Gen5_1 / 60 / 0.5
```

The `_S_` in the SKU marks serverless, and the auto-pause fields now hold values. After 60 idle minutes `az sql db show --query status -o tsv` will read `Paused` (compute billing stops); the next connection auto-resumes it in seconds. To go back to always-on, run the update with `--compute-model Provisioned`.

Goal	Command
See compute model + auto-pause	<code>az sql db show -g <rg> -s <server> -n appdb --query "{sku:currentServiceObjectiveName, autoPause:autoPauseDelay, minCap:minCapacity}" -o table</code>
See serverless flags	<code>az sql db update --help</code>
Convert to serverless	<code>az sql db update -g <rg> -s <server> -n appdb --compute-model Serverless -e GeneralPurpose -f Gen5 --capacity 1 --min-capacity 0.5 --auto-pause-delay 60</code>
Check paused/online status	<code>az sql db show -g <rg> -s <server> -n appdb --query status -o tsv</code>
Back to always-on	<code>az sql db update -g <rg> -s <server> -n appdb --compute-model Provisioned</code>

Part 3 — Practice Exercises

Practice Exercises — Storage and Databases

These are practice problems: **questions only** — no `setup.bash`, no `validate.bash`, no `solution.md`. The graded questions walked you through the core skills; these are the reps that turn understanding into muscle memory.

Work them the same way a cloud admin works any ticket:

1. **Read the scenario into a checklist** of concrete constraints.
2. **Find the reference** — guess `az <group> <verb>`, confirm in the docs.
3. **Tame the flags with `--help`** — required first, then match the rest to your checklist.
4. **Build the command incrementally**, then **verify with your own eyes**.

Some questions deliberately don't hand you the exact flag or value. Finding it in the docs is part of the exercise.

How to check your own work (no validator here)

You are the validator now. The commands that tell you the truth:

```
# --- Blob storage ---
az storage account show -g <rg> -n <acct> -o jsonc                # full definition
az storage account show -g <rg> -n <acct> --query <jmespath> -o tsv
az storage blob list --account-name <acct> -c <container> --auth-mode key -o table
az storage blob show --account-name <acct> -c <container> -n <blob> --auth-mode key -o jsonc
curl -i "<blob-or-sas-url>"                                     # anonymous / SAS access

# --- Azure SQL ---
az sql db list -g <rg> -s <server> -o table                       # all databases on a server
az sql db show -g <rg> -s <server> -n <db> --query <jmespath> -o tsv
az sql server firewall-rule list -g <rg> -s <server> -o table
sqlcmd -S <server>.database.windows.net -U <login> -P '<pw>' -d <db> -Q "SELECT ..."
```

Ground rules

- Use your **own resource groups** — a good convention is `practice-blobNN-rg` and `practice-sqlNN-rg` so they're easy to find and delete. One group per exercise.
- These create **real, billable resources**. Clean up the moment you finish each:

```
az group delete --name <your-rg> --yes --no-wait
```

- **Storage account** and **SQL server** names are globally unique and lowercase.
- For the SQL exercises: your subscription needs the `Microsoft.Sql` provider registered (`az provider`

```
register --namespace Microsoft.Sql --wait), a firewall rule for your client IP, and sqlcmd (pre-installed in Azure Cloud Shell).
```

Part A — Blob Storage

Exercise 1 — Pick your redundancy

Finance wants durable backups that survive a whole datacenter going down, not just a disk failure.

Done when:

- A storage account exists on a **zone-redundant** SKU (find the right `--sku` value; it is not the default).
- It holds a container named `backups`.
- Confirm the SKU with `az storage account show --query sku.name`.

Exercise 2 — Upload a web page as a blob

You are handed `index.html` and asked to store it so that, if served, a browser would render it as a page rather than download it as text.

Done when:

- A container `site` holds a blob `index.html`.
- The blob's **content type** is `text/html` (not the default `application/octet-stream`). Set it at upload time.
- The blob carries **metadata** `owner=<your-name>`.
- Verify both with `az storage blob show ... --query "{type:properties.contentSettings.contentType, meta:metadata}"`.

Exercise 3 — Hand out time-limited access with a SAS

A partner needs to read one file for the next hour without getting a login on your account. A **Shared Access Signature (SAS)** is the tool: a signed, expiring URL.

Done when:

- You generate a **read-only** SAS for a single blob that expires in about 1 hour.
- `curl -i "<blob-url>?<sas>"` returns **HTTP 200** and the content.
- `curl -i "<blob-url>"` (no SAS) is refused, proving the file is not public.
- (Look at `az storage blob generate-sas`; note the `--permissions` and `--expiry` flags, and that you build the full URL as `<blob-url>?<sas>`.)

Exercise 4 — Turn on soft delete and recover a blob

Someone will eventually delete the wrong blob. **Soft delete** keeps deleted blobs recoverable for a retention window.

Done when:

- Blob **soft delete** is enabled on the account with a retention of 7 days.
- You upload a blob, delete it, and confirm it is gone from a normal listing.
- You **restore** it and confirm it is back.
- (Explore `az storage account blob-service-properties delete-policy` to enable it, then `az storage blob delete, list --include d`, and `az storage blob undelete`.)

Exercise 5 — Automate tiering with a lifecycle policy

Manually re-tiering blobs does not scale. A **lifecycle management policy** does it on a schedule.

Done when:

- The account has a lifecycle policy that **moves blobs to Cool after 30 days** since last modification and **deletes them after 365 days**.
 - Confirm with `az storage account management-policy show`.
 - (The policy is JSON. Read `az storage account management-policy create --help` and the docs for the rule shape: filters plus `tierToCool` and `delete` actions.)
-

Part B — Azure SQL Database

Exercise 6 — Add a second database at a different tier

The reporting team wants its own database, sized larger than `appdb`, on the same server.

Done when:

- A second database **reporting** exists on your server.
- It is on the **Standard S1** performance level (20 DTU).
- Confirm with `az sql db list -g <rg> -s <server> -o table` and `az sql db show ... --query "{tier:edition, objective:currentServiceObjectiveName}"`.

Exercise 7 — Create a table and put real data in it

So far you have managed the database from the outside. Now work *inside* it with `sqlcmd`.

Done when:

- Connected to a database with `sqlcmd`, you create a table (for example `customers(id INT, name NVARCHAR(50))`).
- You insert at least three rows.
- `SELECT COUNT(*) FROM customers;` returns your row count.
- (This is pure T-SQL over the connection you opened in the firewall question. Use `-Q "<sql>"` for one-shot statements, or run `sqlcmd` interactively.)

Exercise 8 — Let Azure services in, and an office range

Your app runs on other Azure resources and must reach the database, and the office network (a CIDR range) should also connect.

Done when:

- A firewall rule allows **Azure services** to connect (the special `0.0.0.0-0.0.0.0` rule; find out why that value means “Azure services”).
- A separate named rule allows an **IP range** you choose (for example `203.0.113.0` to `203.0.113.255`).
- Confirm with `az sql server firewall-rule list -g <rg> -s <server> -o table`.

Exercise 9 — Recover with point-in-time restore

A bad `UPDATE` just corrupted data. Azure SQL keeps continuous backups, so you can restore the database to how it looked minutes ago, into a **new** database.

Done when:

- You restore your database to a new database named `appdb-restored`, as of a few minutes ago.

- The new database shows up in `az sql db list` and reports status **Online**.
- (See `az sql db restore`; note `--dest-name` and `--time` in ISO 8601. The earliest restore point is a few minutes after the source database was created.)

Exercise 10 — Capstone: export the database to Blob Storage

Combine both halves of this lab. Export a database to a portable `.bacpac` file and land it in a blob container: exactly how an admin takes a movable backup.

Done when:

- A storage account and a container (for example `exports`) exist.
- You export a database to `<container>/appdb.bacpac` in that account.
- The `.bacpac` blob appears in `az storage blob list`.
- (Look at `az sql db export`; it needs the server admin login and password, plus a storage **account key or SAS** and the target blob URL. The export runs server-side and can take a few minutes. This is a stretch: read the help carefully and build the command one flag at a time.)